

8장 성능 최적화

8.1 성능 최적화

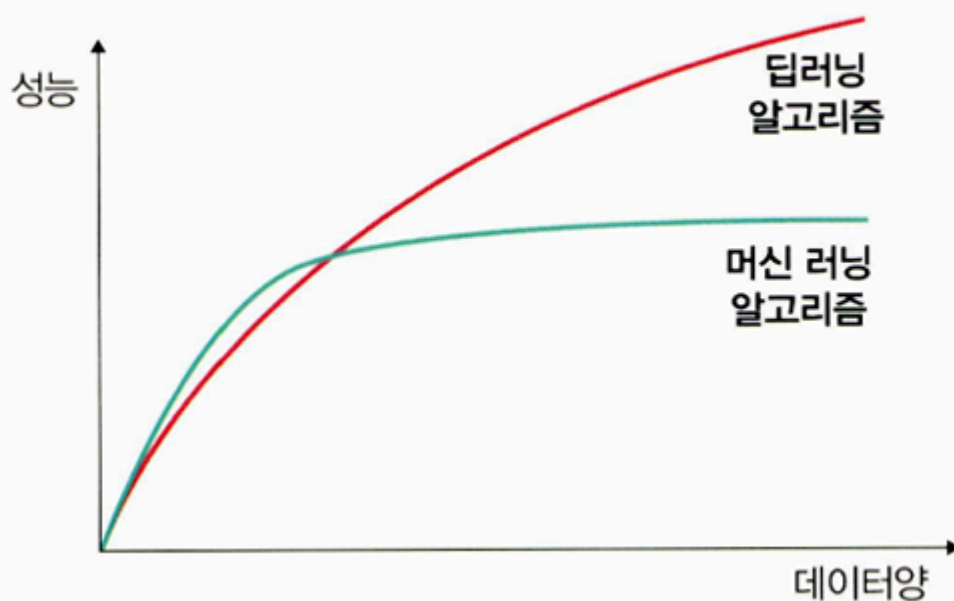
8.1.1 데이터를 사용한 성능 최적화

- 최대한 많은 데이터 수집하기

딥러닝 및 머신 러닝 알고리즘은 데이터량이 많을수록 성능 향상

→ 가능한 많은 데이터(빅데이터) 수집 필요

▼ 그림 8-1 데이터와 딥러닝, 머신 러닝 알고리즘의 성능 비교



- 데이터 생성하기

충분한 데이터를 수집할 수 없는 경우, 데이터를 만들어 사용

→ 5장 이미지 조작 및 코드 참고

- 데이터 범위 조정하기 (scale)

- 시그모이드 사용 시 → 0~1 범위
- 하이퍼볼릭 탄젠트 사용 시 → -1~1 범위

- 정규화, 규제화, 표준화 → 성능 향상에 도움

8.1.2 알고리즘을 이용한 성능 최적화

- 유사한 용도의 알고리즘들을 선정해 직접 모델을 훈련시켜보고, 성능이 가장 좋은 알고리즘을 선택하는 것이 좋음
- 예시:
 - 데이터 분류 → SVM, K-최근접 이웃 등 비교
 - 시계열 데이터 → RNN, LSTM, GRU 등 비교
 - 각 알고리즘을 훈련시켜 성능이 가장 좋은 모델 선택

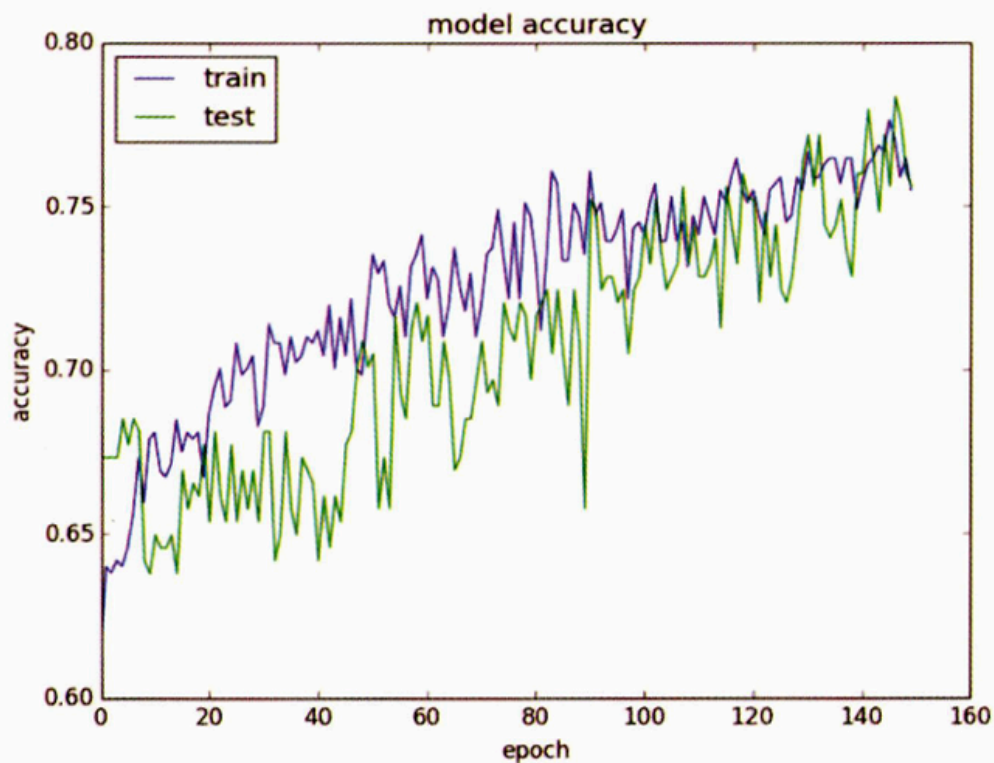
8.1.3 알고리즘 튜닝을 위한 성능 최적화

1. 진단

성능 향상이 멈췄다면 원인 분석 필요

→ 모델 성능 평가 결과를 통해 과적합(overfitting) 또는 다른 문제에 대한 인사이트 확보

▼ 그림 8-2 알고리즘 성능 진단



예시 상황:

- 훈련 성능은 높고 검증 성능은 낮은 경우 → 과적합 의심 → 규제화로 개선
- 훈련·검증 모두 성능 낮은 경우 → 과소적합(underfitting) 의심 → 네트워크 구조 변경 또는 학습 시간/에포크 증가 필요
- 훈련 성능이 기준을 넘었는데 성능 향상 없음 → 조기 종료(early stopping) 고려

1. 가중치 초기화

- 작은 난수를 사용해 초기화
- 오토인코더 학습 결과 등 사전 훈련을 통한 초기화도 가능

2. 학습률

- 초기엔 작은 학습률 사용
- 결과를 보며 점진적 조정
- 계층이 많으면 학습률 증가 필요, 계층이 적으면 작게 설정

3. 활성화 함수

- 활성화 함수 변경 시 손실 함수도 함께 고려
- 데이터 유형과 문제에 따라 적절한 함수 선택
- 일반적으로 입력층에는 시그모이드, 은닉층에는 하이퍼볼릭 탄젠트, 출력층에는 소프트맥스나 시그모이드 자주 사용

4. 배치와 에포크

- 작은 배치보단 적절한 크기의 미니배치 사용
- Adam, RMSProp 등 최적화 알고리즘과 병행해 테스트 후 결정

5. 네트워크 구성

- 구성은 topology라고도 함
- 은닉층 수, 뉴런 수 등 실험적으로 조정
 - 예: 은닉층에 뉴런 여러 개 포함 → 네트워크 폭 확장

- 은닉층 수 늘리기 → 네트워크 깊이 증가
- 두 가지 모두 실험 후 가장 성능 좋은 구성 선택

8.1.4 앙상블을 이용한 성능 최적화

- 앙상블

: 두 개 이상의 모델을 결합하여 사용하는 것

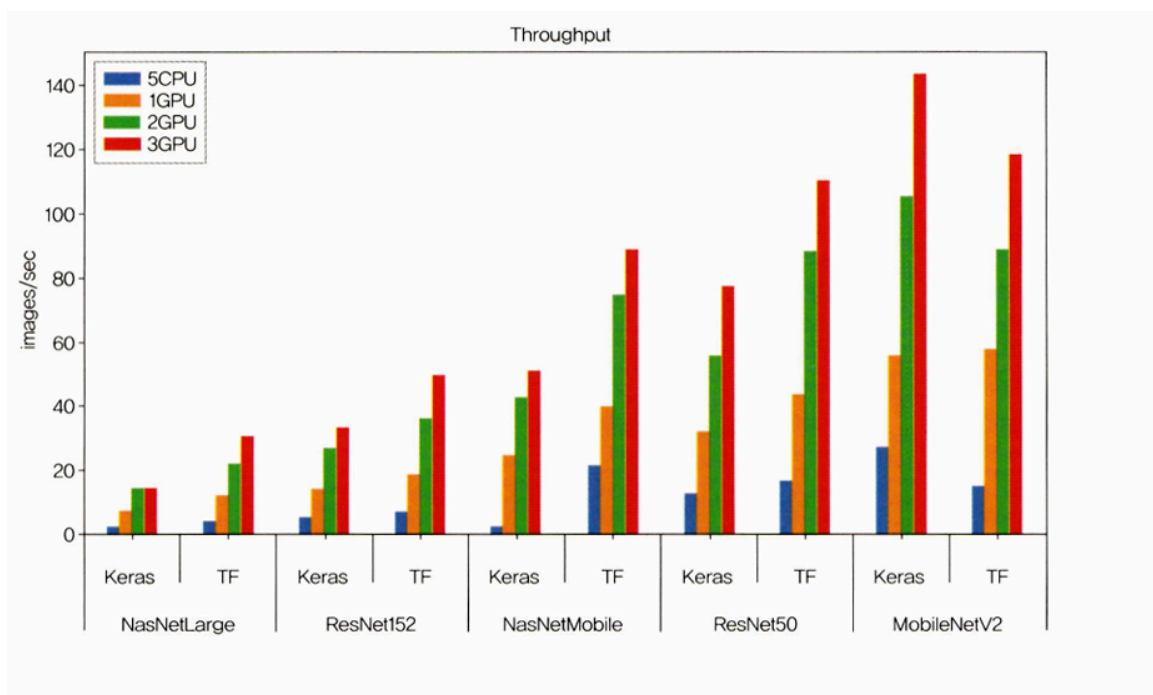
→ 성능 향상에 효과적

8.2 하드웨어를 이용한 성능 최적화

CPU 와 GPU 사용의 차이

CPU와 GPU 성능 비교

- 딥러닝에서 GPU를 사용하면 분석 시간이 단축됨



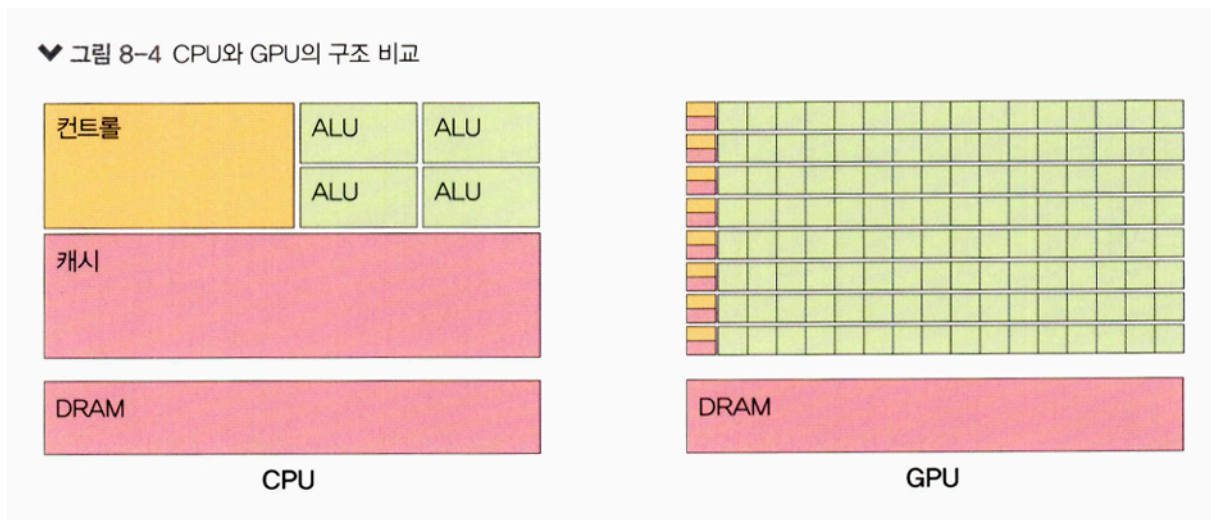
- CPU 여러 개보다 GPU 하나가 성능 더 좋음

→ 이유: CPU와 GPU의 내부 구조 차이

CPU 구조 특징

GPU 구조 특징

- 명령어를 순차적으로 처리
- 구성: 제어장치(Control), 캐시(Cache), 소수의 연산 유닛(ALU)
- 병렬 연산에 적합하지 않음
- 복잡한 명령을 처리하되, 순차 처리 방식 → 속도 한계 있음
- 병렬 연산에 특화
- 많은 수의 ALU로 구성
- 메모리 캐시 작고, 병렬 처리에 최적화
- 수천 개 코어가 병렬 연산을 수행
- 행렬 연산, 3D 그래픽, 딥러닝 등에 적합
- 속도 측면에서 CPU보다 빠름



GPU를 이용한 성능 최적화

윈도우 환경에서 GPU 용의 파이토치를 설치하려면 CUDA 와 cuDNN 을 설치해야 한다.

CUDA

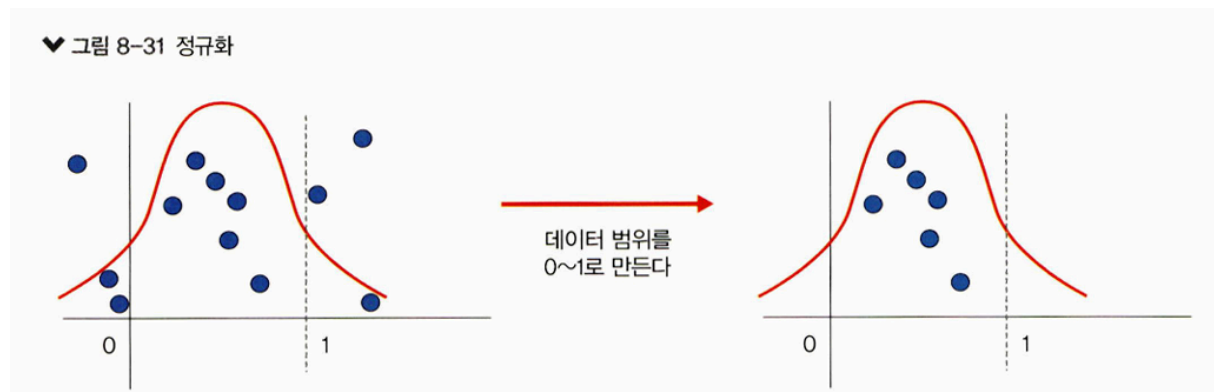
엔비디아에서 개발한 GPU 개발 툴로, 많은 양의 연산을 동시에 처리할 수 있게 해준다.

8.3 하이퍼파라미터를 이용한 성능 최적화

배치 정규화를 이용한 성능 최적화

정규화

데이터 범위를 사용자가 원하는 범위로 제한하는 것



- 특성 스케일링 (feature scaling) 이라고도 함

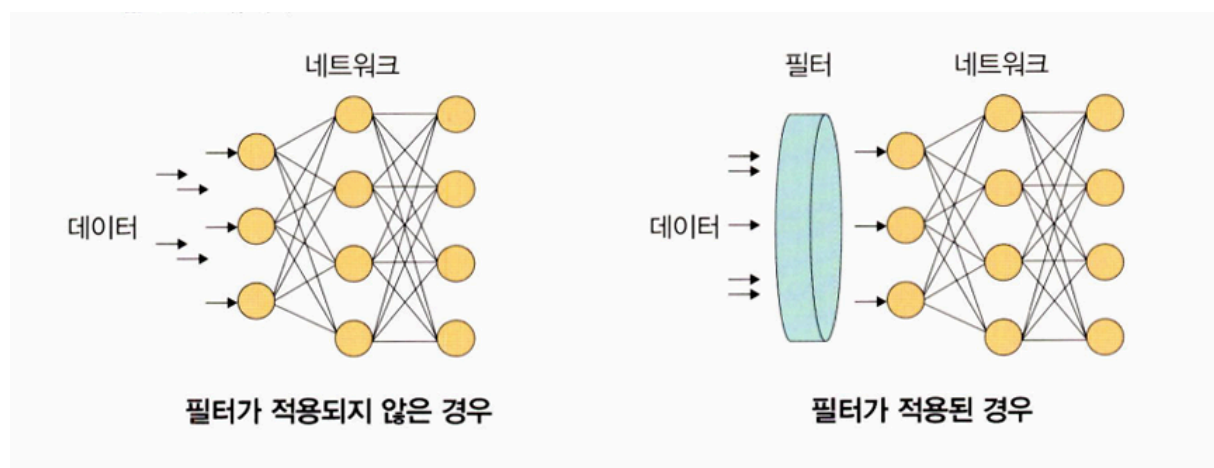
$$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

(x : 입력 데이터)

규제화

모델 복잡도를 줄이기 위해 제약을 두는 방법. 데이터가 네트워크에 들어가기 전에 필터를 적용한다.

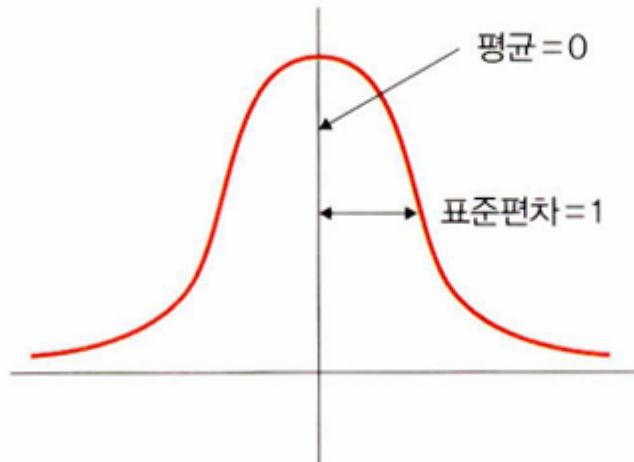
ex) 드롭아웃, 조기 종료



표준화 = 표준화 스칼라, z-스코어 정규화

기존 데이터를 평균은 0, 표준편차는 1인 형태의 데이터로 만드는 방법

▼ 그림 8-33 표준화



- 수식

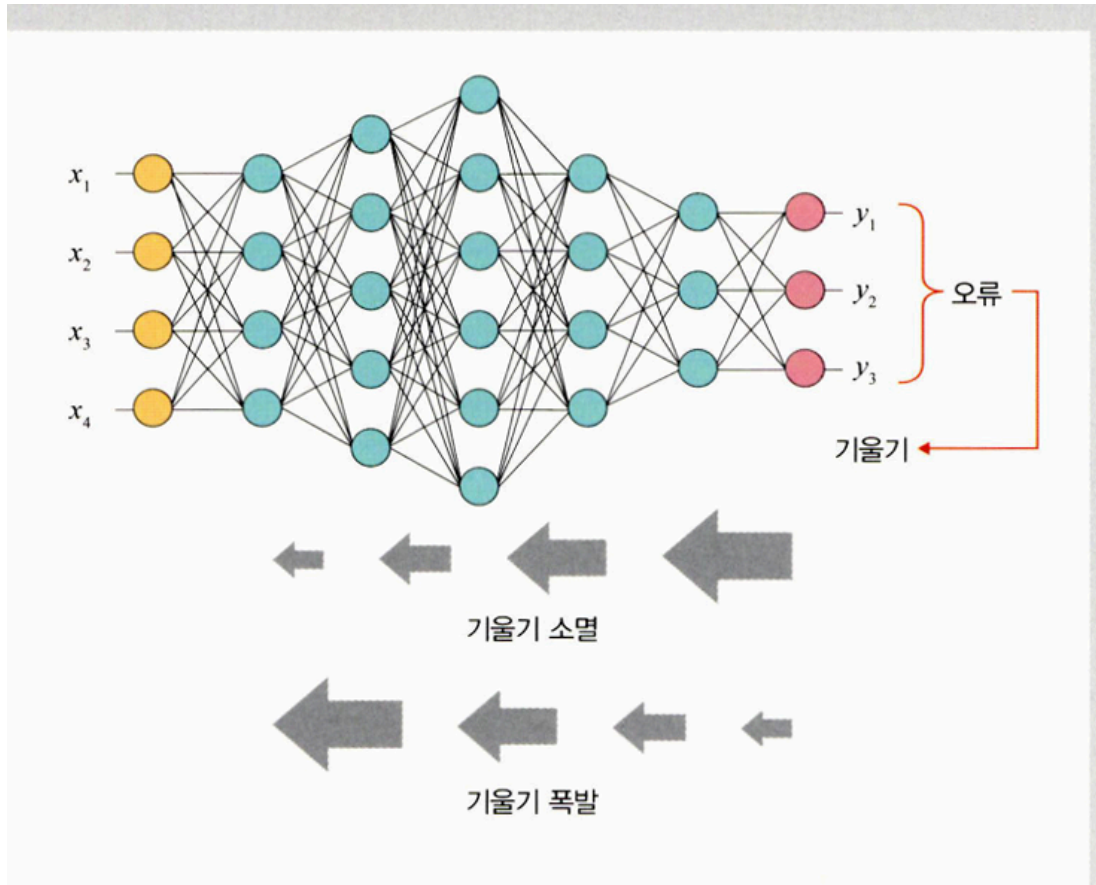
$$\frac{x - m}{\sigma}$$

(σ : 표준편차, x : 관측 값(입력 값), m : 평균)

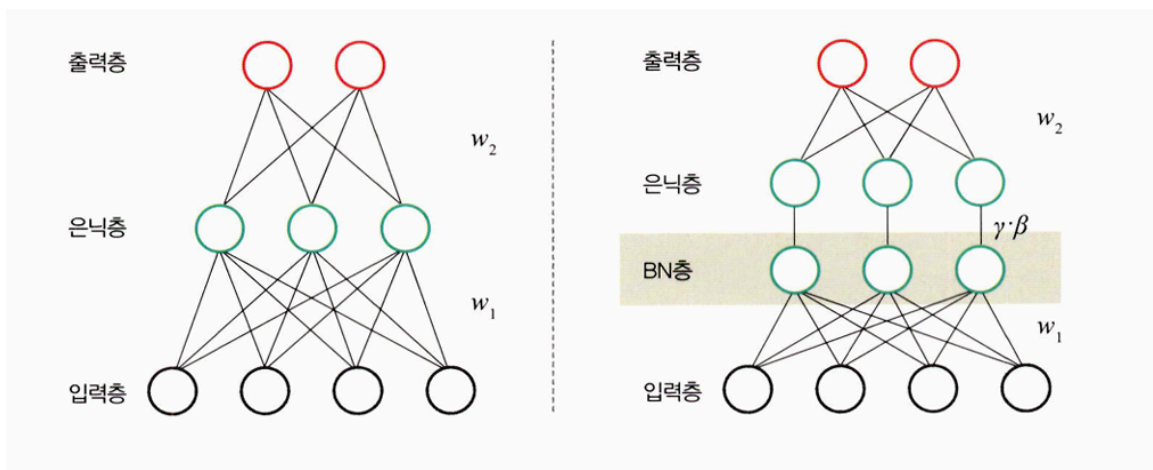
- 보통 데이터 분포가 가우시안 분포를 따를 때 유용한 방법으로 다음 수식을 사용한다.

배치 정규화

- 기울기 소멸이나 기울기 폭발과 같은 문제를 해결하기 위한 방법
 - 기울기 소멸 : 오차 정보를 역전파시키는 과정에서 기울기가 급격히 0에 가까워져 학습이 되지 않음
 - 기울기 폭발 : 학습 과정에서 기울기가 급격히 커지는 현상



- 손실 함수로 렐루를 사용하거나 초깃값 튜닝, 학습률 등 조정



- 기울기 소멸과 폭발의 원인은 내부 공변량 변화 때문인데, 이것은 네트워크의 각 층마다 활성화 함수가 적용되면서 입력 값들의 분포가 계속 바뀌는 현상을 의미한다.
→ 분산된 분포를 정규분포로 만들기 위해 표준화와 유사한 방식을 미니 배치에 적용하여 평균은 0으로, 표준편차는 1로 유지하도록 함

$$\begin{aligned}\mu\beta &\leftarrow \frac{1}{m} \sum_{i=1}^m x_i & \cdots \cdots ① \\ \sigma^2\beta &\leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu\beta)^2 & \cdots \cdots ② \\ \hat{x}_i &\leftarrow \frac{x_i - \mu\beta}{\sqrt{\sigma^2\beta + \epsilon}} & \cdots \cdots ③ \\ y_i &\leftarrow \gamma \hat{x}_i + \beta \Leftrightarrow BN_{\gamma, \beta}(x_i) & \cdots \cdots ④ \\ \left(\begin{array}{l} \text{입력: } \beta = \{x_1, x_2, \dots, x_n\} \\ \text{학습해야 할 하이퍼파라미터: } \gamma, \beta \\ \text{출력: } y_i = BN_{\gamma, \beta}(x_i) \end{array} \right)\end{aligned}$$

- ① 미니 배치 평균을 구합니다.
- ② 미니 배치의 분산과 표준편차를 구합니다.
- ③ 정규화를 수행합니다.
- ④ 스케일(scale)을 조정(데이터 분포 조정)합니다.

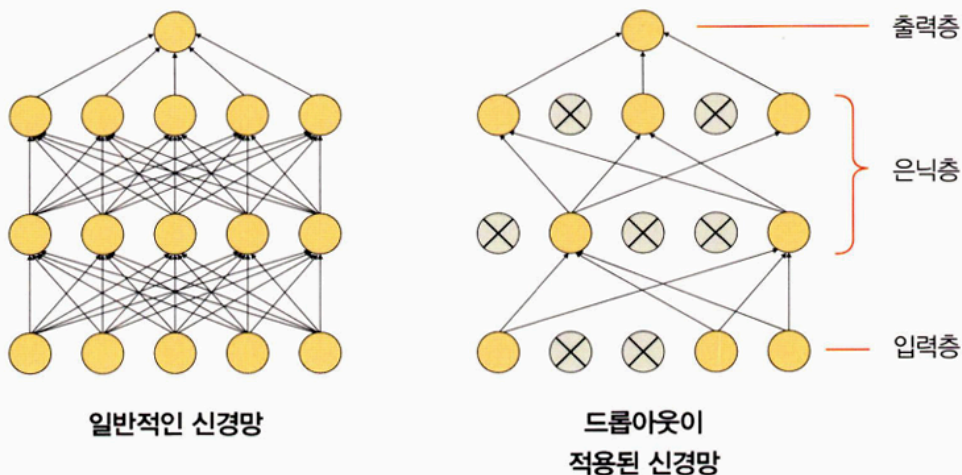
⇒ 매 단계마다 활성화 함수를 거치면서 데이터셋 분포가 일정해지기 때문에 속도를 향상시킬 수 있지만

1. 배치 크기가 작을 때는 정규화 값이 기존 값과 다른 방향으로 훈련될 수 있음
2. RNN 은 네트워크 계층별로 미니 정규화를 적용해야 하기 때문에 모델이 더 복잡해지면서 비효율적일 수 있음

드롭아웃을 이용한 성능 최적화

훈련할 때 일정 비율의 뉴런만 사용하고 나머지 뉴런에 해당하는 가중치는 업데이트 하지 않는 방법으로, 매 단계마다 사용하지 않는 뉴런을 바꾸어 가며 훈련.

▼ 그림 8-37 드롭아웃



8-4. 데이터셋 분리

```
# 예제 8-4 데이터셋 분리
dataiter = iter(trainloader)
images, labels = next(dataiter)

print(images.shape)    # 전체 배치 이미지 텐서의 shape 출력
print(images[0].shape) # 첫 번째 이미지의 shape 확인
print(labels[0].item()) # 첫 번째 이미지의 레이블 값 출력
```

`torch.Size([4, 1, 28, 28])`

ⓐ ⓑ ⓒ

출력 크기

- a. 한 번의 배치 크기로 몇 개의 데이터를 가져오는지 의미
- b. 채널을 의미하는 것으로 흑백 이미지는 1, 컬러 이미지는 3
- c. 28×28 픽셀 크기의 이미지라는 의미

6-8. 배치 정규화가 적용되지 않은 네트워크

```
# 예제 8-8 배치 정규화가 적용되지 않은 네트워크
class NormalNet(nn.Module):
    def __init__(self):
        super(NormalNet, self).__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48), # (28, 28) 크기 이미지 → 784차원 입력
            nn.ReLU(),
            nn.Linear(48, 24),
            nn.ReLU(),
            nn.Linear(24, 10) # 출력: FashionMNIST의 10개 클래스
        ) # nn.Sequential로 계층을 순차적으로 구성

    def forward(self, x):
```

```
x = x.view(x.size(0), -1) # 2D 이미지를 1D 벡터로 변환 (batch_size, 784)
x = self.classifier(x)    # 정의된 계층(classifier) 통과
return x
```

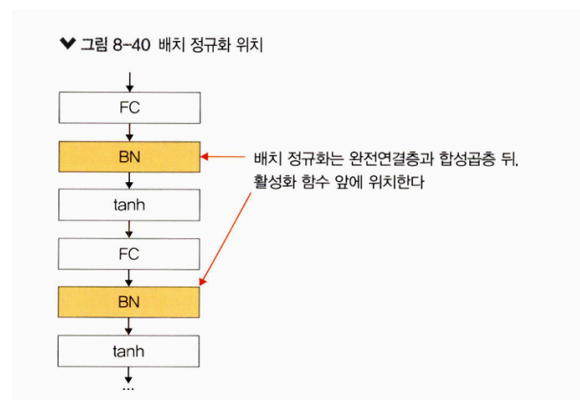
6-9. 배치 정규화가 적용된 네트워크

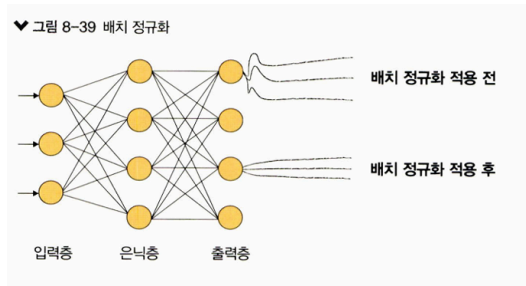
예제 8-9 배치 정규화가 포함된 네트워크

```
class BNNet(nn.Module):
    def __init__(self):
        super(BNNet, self).__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48),      # 입력: 28x28 = 784
            nn.BatchNorm1d(48),      # 배치 정규화 → 출력 채널 수와 동일하게 설정
            nn.ReLU(),
            nn.Linear(48, 24),
            nn.BatchNorm1d(24),      # 배치 정규화 추가
            nn.ReLU(),
            nn.Linear(24, 10)        # FashionMNIST 10개 클래스 분류
        )

    def forward(self, x):
        x = x.view(x.size(0), -1)  # 이미지 텐서를 펼쳐서 (batch_size, 784)로 변환
        x = self.classifier(x)      # Sequential에 정의된 계층 적용
        return x
```

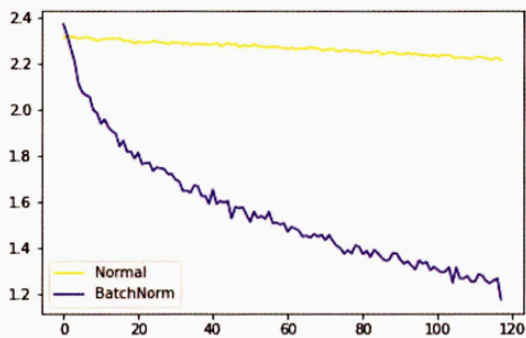
*** 배치 정규화를 사용하는 이유 :** 은닉층에서 학습이 진행될 때마다 입력 분포가 변하면서 가중치가 엉뚱한 방향으로 갱신되는 문제를 해결하기 위해



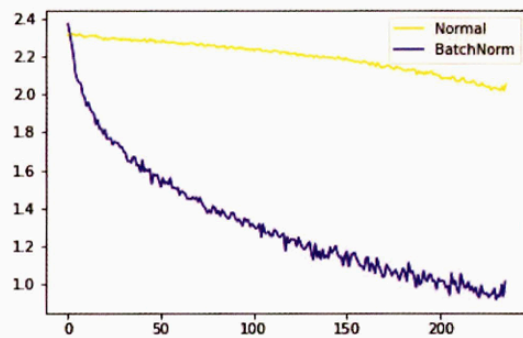


모델 학습 결과

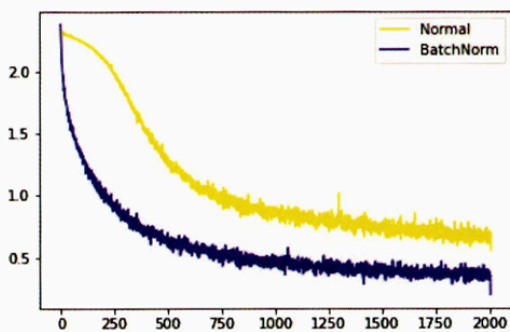
▼ 그림 8-41 첫 번째 에포크에 대한 학습 결과



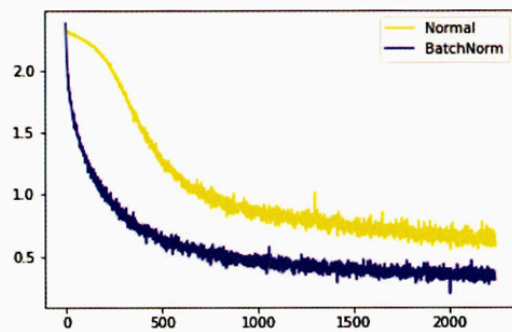
▼ 그림 8-42 두 번째 에포크에 대한 학습 결과



▼ 그림 8-43 18번째 에포크에 대한 학습 결과



▼ 그림 8-44 19번째 에포크에 대한 학습 결과



→ 둘 다 시간이 지날수록 오차가 줄어들지만 오차가 줄어드는 범위 및 값의 차이가 명확하다.

테스트 데이터셋의 분포

예제 8-15 데이터셋의 분포를 출력하기 위한 전처리

```
N = 50                # 데이터 포인트 수
noise = 0.3           # 잡음 비율
```

```
# x_train: -1 ~ 1 사이의 값 50개를 1열로 구성한 텐서
x_train = torch.unsqueeze(torch.linspace(-1, 1, N), 1)
```

```
# y_train: x_train에 정규분포 잡음을 추가
y_train = x_train + noise * torch.normal(torch.zeros(N, 1), torch.ones(N, 1))
```

```
# 테스트 데이터셋도 동일 방식 생성
x_test = torch.unsqueeze(torch.linspace(-1, 1, N), 1)
y_test = x_test + noise * torch.normal(torch.zeros(N, 1), torch.ones(N, 1))
```

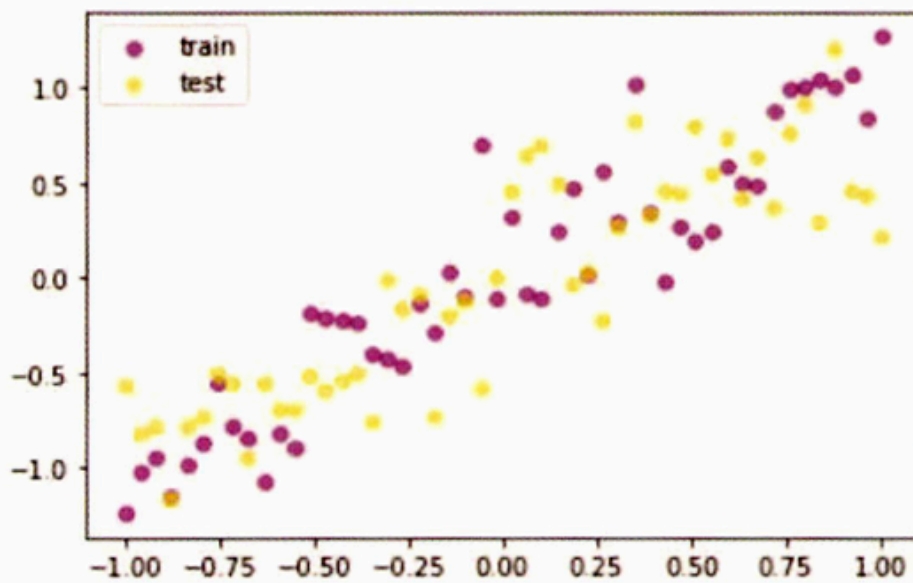
예제 8-16 데이터 분포를 그래프로 출력

```
plt.scatter(x_train.data.numpy(), y_train.data.numpy(), c='purple',
            alpha=0.5, label='train') # 학습 데이터 분포 시각화
```

```
plt.scatter(x_test.data.numpy(), y_test.data.numpy(), c='yellow',
            alpha=0.5, label='test') # 테스트 데이터 분포 시각화
```

```
plt.legend()
plt.show()
```

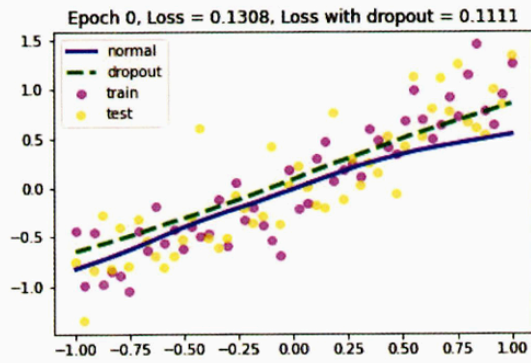
▼ 그림 8-45 FashionMNIST 데이터셋의 분포



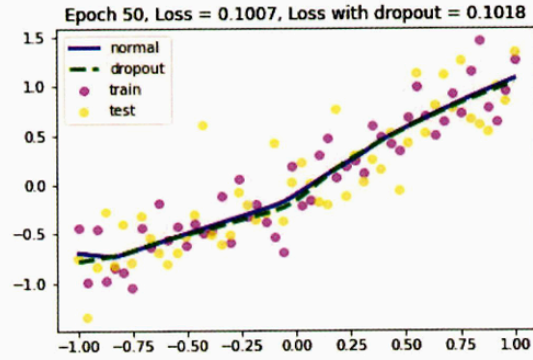
→ 훈련과 테스트 데이터가 고르게 분포되어 있는 것을 알 수 있다.

드롭아웃과 드롭아웃을 적용하지 않은 모델의 학습 결과 비교

▼ 그림 8-46 첫 번째 드롭아웃에 대한 학습 결과

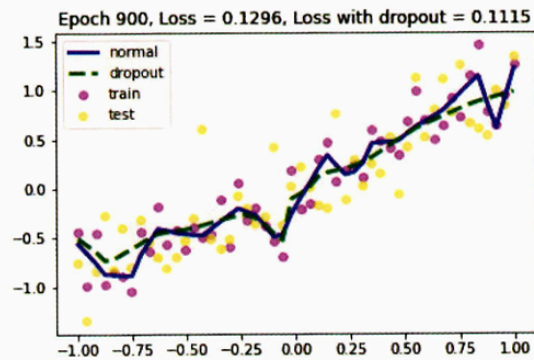


▼ 그림 8-47 두 번째 드롭아웃에 대한 학습 결과

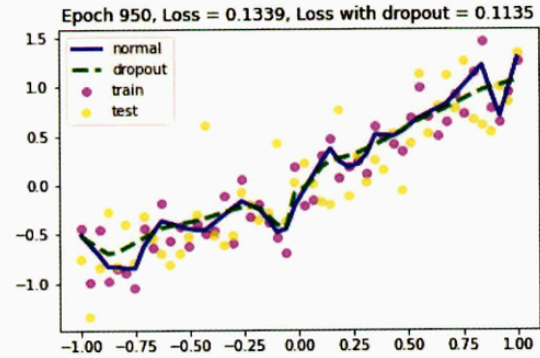


... 중간 생략 ...

▼ 그림 8-48 19번째 드롭아웃에 대한 학습 결과



▼ 그림 8-49 20번째 드롭아웃에 대한 학습 결과



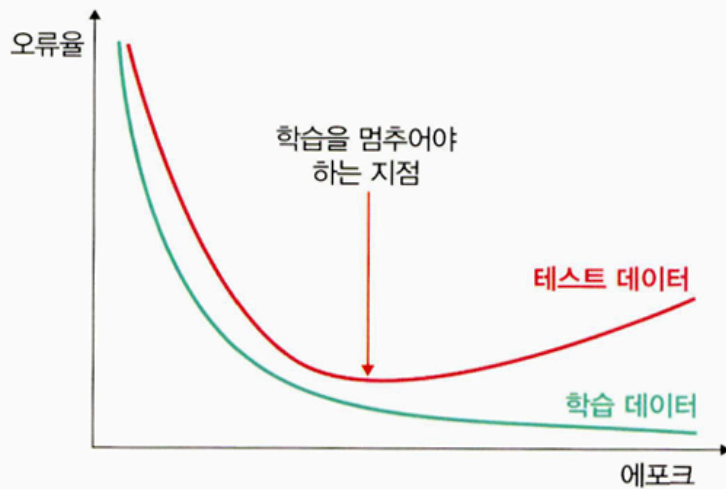
→ 전반적으로 큰 차이는 없지만 드롭아웃을 적용했을 때의 오차가 더 낮다.

조기 종료를 이용한 성능 최적화

훈련 데이터와 별도로 검증 데이터를 준비하고 매 에포크마다 검증 데이터에 대한 오차를 측정하여 모델의 종료 시점 제어

⇒ 검증 데이터셋에 대한 오차가 증가하는 시점에서 학습을 멈추도록 조정하는 기법

♥ 그림 8-50 조기 종료



코드 실습

8.24) 학습률 감소

#학습 과정에서 모델 성능에 대한 개선이 없을 경우 학습률 값을 조정하여 모델의 개선을 !

```
class LRScheduler():
```

```
    def __init__(self, optimizer, patience=5, min_lr=1e-6, factor=0.5):
```

```
        self.optimizer = optimizer
```

```
        self.patience = patience
```

```
        self.min_lr = min_lr
```

```
        self.factor = factor
```

```
        self.lr_scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
```

```
            self.optimizer,      # 옵티마이저 객체
```

```
            mode='min',          # val_loss 기준으로 최소값 추적
```

```
            patience=self.patience,
```

```
            factor=self.factor,
```

```
            min_lr=self.min_lr,
```

```
            verbose=True          # 줄어들 때마다 로그 출력
```


)

```
def __call__(self, val_loss):  
    self.lr_scheduler.step(val_loss) # 검증 손실 기준으로 step 진행
```

- 학습률에 대한 값을 고정시켜서 모델을 학습시키는 것이 아니라 학습이 진행되는 과정에서 학습률을 조금씩 낮추어 주는 성능 튜닝 기법 중 하나.
 - 주어진 'patience' 횟수만큼 검증 데이터셋에 대한 오차 감소가 없으면
 - 주어진 'factor' 만큼 학습률을 감소시켜서 모델 학습의 최적화가 가능하도록 함

- `LRScheduler()` 클래스

항목	설명
optimizer	학습 파라미터 최적화 도구 (예: <code>optim.Adam</code>)
mode	기준 지표가 작아야 하는지(<code>min</code>) 커야 하는지 설정→ <code>val_loss</code> 기준이면 ' <code>min</code> ', <code>val_acc</code> 기준이면 ' <code>max</code> '
patience	학습률을 줄이기 전 기다리는 에폭 수→ 예: <code>5</code> 로 설정 시 5 번 동안 성능 개선 없으면 감소
factor	학습률 감소 배율 (예: <code>0.5</code> 면 절반으로 줄임)
min_lr	학습률의 최솟값 (그 이하로는 줄어들지 않음)
verbose	학습률 감소 시 로그 출력 여부 (<code>1</code> 이면 출력, <code>0</code> 이면 무출 력)

- `__call__` 클래스 : 실제로 학습률을 업데이트

8-24. 조기 종료

```
class EarlyStopping():  
    def __init__(self, patience=5, verbose=False, delta=0, path='./chap08/data/c  
        self.patience = patience  
        #오차 개선이 없는 에포크를 얼마나 기다려 줄지 지정  
        self.verbose = verbose  
        self.counter = 0  
        self.best_score = None  
        self.early_stop = False  
        self.val_loss_min = np.inf
```

```

self.delta = delta
#오차가 개선되고 있다고 판단하기 위한 최소 변화량
self.path = path

def __call__(self, val_loss, model):
    score = -val_loss

    if self.best_score is None:
        self.best_score = score
        self.save_checkpoint(val_loss, model)

    elif score < self.best_score + self.delta:
        self.counter += 1
        print(f"EarlyStopping counter: {self.counter} out of {self.patience}")
        if self.counter >= self.patience:
            self.early_stop = True

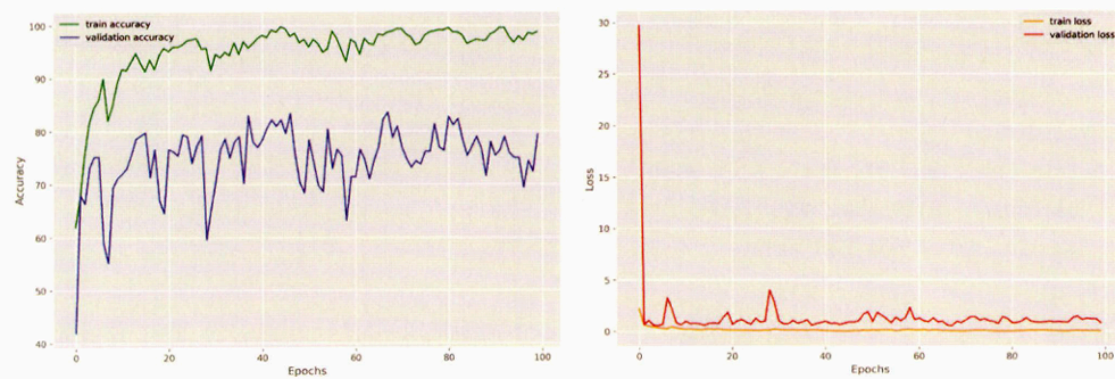
    else:
        self.best_score = score
        self.save_checkpoint(val_loss, model)
        self.counter = 0

def save_checkpoint(self, val_loss, model):
    if self.verbose:
        print(f'Validation loss decreased ({self.val_loss_min:.6f} → {val_loss:.6f})')
    torch.save(model.state_dict(), self.path)
    self.val_loss_min = val_loss

```

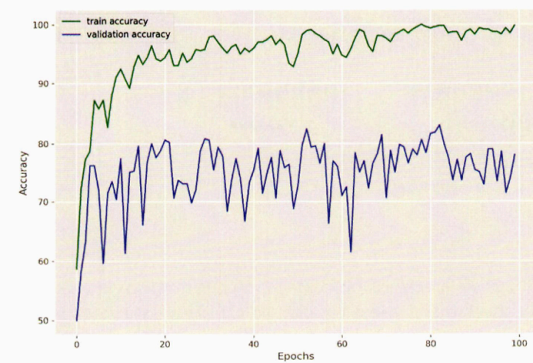
출력 결과

▼ 그림 8-53 출력 결과

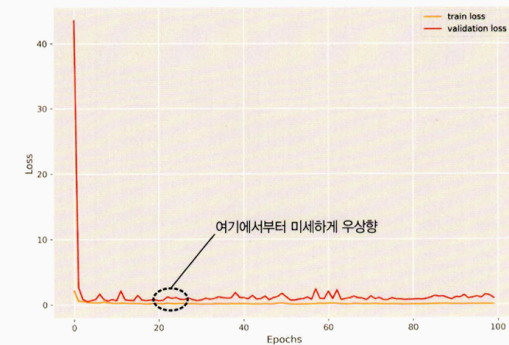


- 어떤 인수도 적용하지 않았을 때

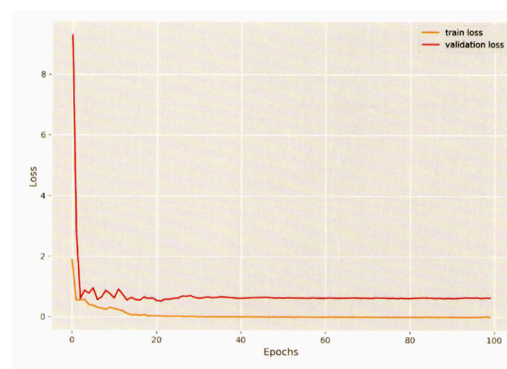
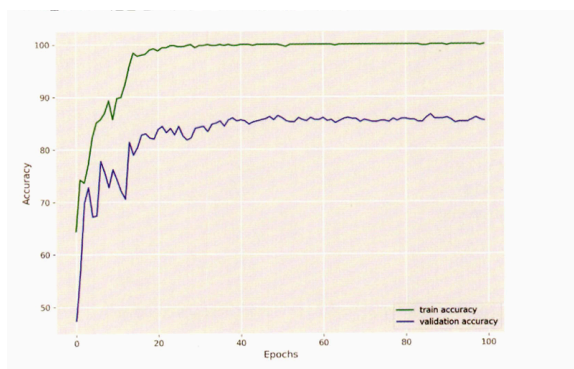
▼ 그림 8-54 어떤 인수도 적용되지 않았을 때의 정확도



▼ 그림 8-55 어떤 인수도 적용되지 않았을 때의 오차

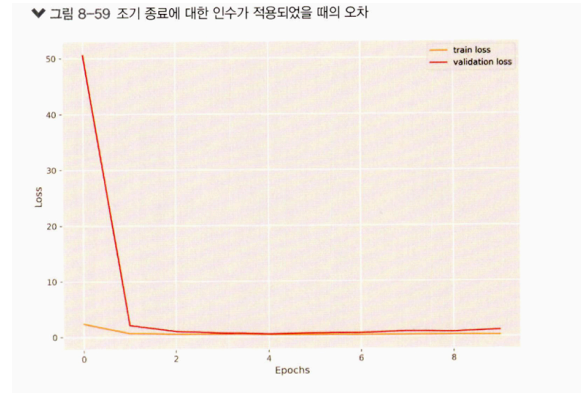
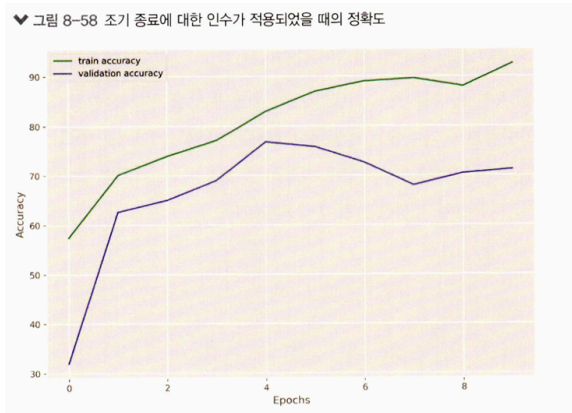


- 학습률 감소와 관련된 인수를 입력했을 때



→ 정확도 그래프가 완만한 곡선 형태를 띄고, 훈련이 종료된 시점의 검증 데이터셋에 대한 정확도도 높게 나타난다.

- 에포크 20 정도에서 val 오차가 정체
- 조기 종료 인수를 적용했을 때



→ 다섯 번째 에포크부터 조기 종료가 수행되어 실제 학습은 네 번째 에포크까지만 수행
 → 검증 데이터셋에 대한 정확도는 좋지 않지만 오차는 많이 낮아짐